BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
26 October 2026

Abstract

Agent tasks need private working state while reusing source trees, dependencies, and datasets. Sharing those bytes avoids duplication, but leaves the cost of serving metadata and partial reads. BrewFS addresses these costs through three independently optional mechanisms. Delta-Based Workspaces retain private namespace changes and data-range mappings over a shared baseline. Packed Metadata serves a selected, previously packed read-only subtree through indexed object pages that co-locate names, attributes, and data locators. It targets stable collections of many small files, including training datasets reused across machine-learning experiments. Variable-Granularity Object Storage separates client object formation from backend encoding and verification units. Each mechanism has a distinct cost: retaining private changes, building and publishing packed versions, or fetching and verifying storage units. We describe their representations, update protocols, and implementation limits. A study of 1,138,884 FUSE requests from eight coding-agent runs finds substantial metadata traffic, mixed request sizes, and bursty access. The planned evaluation tests whether savings repay preparation and retention costs, then examines their effect on fork, migration, and concurrent execution.

Draft status. The workload characterization is complete. System-performance experiments and end-to-end recovery validation remain to be conducted.

1 Introduction

Coding agents traverse repositories, inspect files, edit code, and run builds and tests [24, 27]. When a service launches several attempts from the same revision, each needs a consistent starting view and private changes. Saved results must remain available for later attempts or another worker. Storage must provide these views while serving their names, attributes, and file data throughout execution.

Machine-learning (ML) agents also prepare datasets, train models, and compare experiments [3]. Consider several such agents exploring models against one fixed training dataset. Their code, checkpoints, and temporary outputs differ, while the input data remains read-only. If the dataset contains many small files, sharing its bytes still leaves a large namespace to serve. This case motivates a selective optimization: pack the stable data directory once and reuse its metadata across experiments, while keeping mutable working state outside the pack.

Prior systems provide much of the foundation. OverlayFS and BranchFS isolate private changes over shared state [13, 22]; DeltaFS publishes private namespace updates as immutable logs [28]. IndexFS and InfiniFS improve metadata organization, distribution, and caching [14, 19]. CernVM-FS and Nydus serve immutable file trees with indexed metadata and on-demand data access [2, 15]. CoFS exploits a fixed image namespace to index metadata for kernel lookup [23]. Building on these approaches, an object-backed workspace must decide how to represent private updates, which fixed metadata to serve outside the mutable database, and which units to fetch and verify.

These decisions incur different costs. In a database-backed file system, queries over fixed collections can compete with active namespace updates [8]. Packing redirects supported queries to object storage and client caches, at the cost of constructing and publishing a version. A native overlay can fork without that additional work. Separately, object formation and backend layout determine how much storage work a partial read requires. Reducing fork cost therefore does not necessarily reduce the cost of using the resulting workspace.

Our workload study reinforces the need to examine access as well as creation. Across 1,138,884 FUSE requests from eight coding-agent runs, metadata methods accounted for 57.3% of all calls. Requests also varied in length and arrived in bursts (Section 2.3). These observations characterize coding workloads; they do not measure ML data access or establish a metadata-service bottleneck.

BrewFS separates the identity of shared state from its metadata representation and payload access units. Three

mechanisms implement this separation:

- **Delta-Based Workspaces** retain private namespace state and data-range mappings over a pinned baseline. Unlike file-level copy-up in the BranchFS prototype, unchanged ranges keep their original references. The lower-query interface accepts native or packed metadata (Section 4).

- **Packed Metadata** serves an explicitly packed read-only subtree through authenticated object pages. Directory-name groups co-locate entries, common attributes, and data locators for reuse across related operations. Fixed-version queries leave the per-file database path; mutable updates and lifetime control remain in their existing services (Section 5).

- **Variable-Granularity Object Storage** separates client object formation from backend encoding and verification. A finalized object can carry a layout hint, which the backend validates and persists for subsequent reads and repair (Section 6).

Each mechanism can be used independently. A workspace can retain native metadata; a packed dataset can serve standalone read-only access; an ordinary volume can configure object granularity. Packing is most plausible for stable, metadata-intensive collections reused across many readers. Frequent updates or little reuse may leave native metadata cheaper, while smaller storage units can add requests and checksums.

This paper presents the design, implementation, and completed workload characterization. The evaluation plan isolates the three mechanisms before examining fork, migration, and concurrent execution (Section 8).

2 Background and Motivation

2.1 Private work and shared inputs

An agent's workspace contains source files, dependencies, generated artifacts, and temporary files. Two attempts starting from the same revision may read the same file and then make incompatible edits. Each must continue to see its own version, including after deletion or truncation. A stable baseline identity and private changes provide this isolation independently of where metadata and bytes reside.

Read-only inputs have a different lifetime from working state. In the ML scenario, an experiment reads a fixed dataset while writing model checkpoints and transformed outputs elsewhere. A directory-based image loader, for example, enumerates a dataset to build a sample list, then opens selected files during loading [21]. Separate experiments can repeat this discovery work against the same version.

Only a selected, already packed directory benefits from Packed Metadata. New files and in-place updates are outside that immutable version; revised data requires publication of a new version. A private upper can hold workspace changes without modifying the packed lower. Small-file collections are a particular target because they require many names, attributes, and data locators relative to their payload size. Datasets stored in a few large shards offer less of this opportunity.

2.2 From file operations to storage work

Reading a file requires path resolution, a data mapping, and access to the referenced bytes. Metadata services batch and cache records [18, 19], while immutable distributions organize them for demand fetching [2, 15]. A file-system call therefore need not cause a remote metadata request. The relevant question is how much database traffic remains under the actual cache and concurrency conditions.

For a fixed small-file collection, object-resident metadata can move read service away from the database handling mutable updates. Construction and publication must be amortized over the version's uses. The likely gain also depends on locality: neighboring queries can share a metadata page, whereas scattered queries may fetch records they never use.

Payload access introduces another translation. Kernel caching and readahead determine which requests reach FUSE; the client then fetches object ranges or complete objects. Encoding and integrity checks may require still larger physical reads [1, 6]. Writes pass through buffering and object formation before publication. FUSE request lengths thus describe neither application syscall sizes nor physical storage units.

2.3 Observed requests in coding-agent runs

Capture. We reanalyzed eight SWE-bench Verified task runs [7]. All used the same harness configured for `deepseek-v4-flash`. Three runs had a broader investigation and testing prompt and a 30-minute budget; all finished earlier. Each base-commit checkout used an instrumented passthrough file system on Ubuntu 24.04 under WSL. Instrumentation recorded requests reaching FUSE handlers.

The capture contains 1,138,884 requests across all eight runs. Counts include all statuses, 64 setup/canary requests before startup, and teardown, including 24 requests after recorded run completion. Pooled statistics weight requests equally rather than tasks. The companion workload record retains instrumentation versions, per-task counts, and source-validation details.

Metadata traffic. Six metadata methods (`lookup`, `getattr`, `getxattr`, `readdirplus`, `opendir`, and

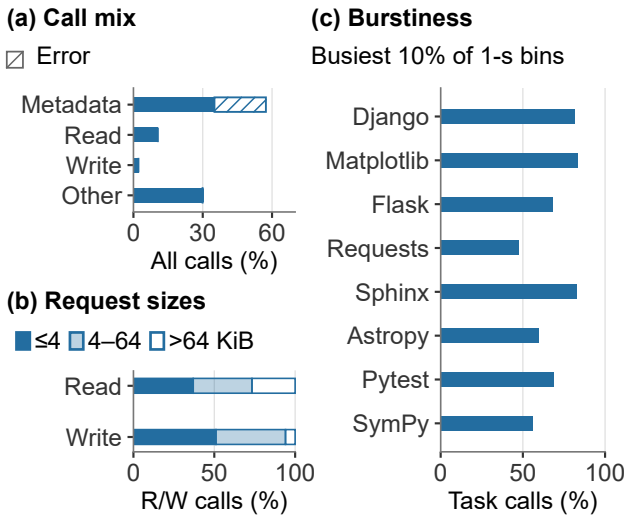


Figure 1: FUSE requests across eight coding-agent runs. (a) Pooled call shares; hatching marks errors within each category. (b) Pooled requested-length distributions, normalized separately for reads and writes. (c) Each task's call share in its busiest 10% of one-second windows. Panels (a,b) weight requests equally.

`releasedir`) accounted for 57.3% of all calls (Figure 1(a)). Other contains methods outside these six and read/write. Among successful events, the metadata share was 45.8%, showing the contribution of unsuccessful probes. Across individual tasks, all-status shares ranged from 39.6% in SymPy to 65.4% in Django. Read calls outnumbered writes by 4.87:1 overall; per-task ratios ranged from 3.12:1 to 42.7:1.

Request sizes. All 120,434 reads and 24,712 writes succeeded. Reads ranged from 4 to 128 KiB, with a median of 12 KiB. Writes ranged from one byte to 128 KiB, with a median of 4 KiB. Requests of at most 4 KiB accounted for 37.0% of reads and 51.0% of writes. The interval $(4, 64]$ KiB contained a further 36.3% of reads and 43.0% of writes (Figure 1(b)). This mixture calls for measuring how fetching and object formation transform requests before selecting storage granularity.

Variation across tasks and time. Tasks generated 8,542–634,726 calls over capture spans of 42.7–556.1 seconds. Astropy contributed 55.7% of all requests, so the pooled distributions describe recorded traffic rather than an equally weighted typical task. The busiest 10% of one-second windows contained 47.4–83.7% of each task's calls (Figure 1(c)). Windows include idle and final partial windows; the number selected is rounded up per task. These differences favor retaining per-task mixes and timing in replay. Each task was observed once, and repositories, prompts, and budgets differed; the observations do not isolate their effects.

Measurement limits. The capture contains 872,519 `ok`, 253,669 `error`, and 12,696 `incomplete` events. Errors came from `lookup` (233,452), `getxattr` (20,203), and `rmdir` (14). Missing errno values prevent attribution of individual errors. Incomplete events lack a recognized paired completion: 12,688 are `forget` records and eight are `destroy` records. All agent and daemon exit codes were zero; task-grading success was not established.

Handler intervals measure WSL passthrough activity, not remote storage latency or model response time. Capture spans include inter-request gaps; overlapping handler intervals cannot be summed into task duration. The traces establish request mix and timing for this panel. They do not measure ML workloads, cross-task sharing, sparse mutation, or BrewFS performance.

2.4 Design implications

The coding traces motivate positive and negative metadata queries, mixed request lengths, and bursty replay. The ML scenario adds a complementary question: can repeated access to a fixed small-file dataset repay packing and reduce interference with mutable work? Both require measuring the work induced below the file-system interface. Section 3 describes how BrewFS separates workspace state, metadata representation, and payload layout to make these costs explicit.

3 System Overview

3.1 State, metadata, and payloads

BrewFS separates three decisions along the file-access path: which version a task sees, how it retrieves immutable metadata, and how it stores and fetches payloads. A lower reader resolves names, attributes, and data locators under a fixed version identity. The data path returns the referenced logical bytes using each object's persisted layout. These interfaces allow metadata representation and object layout to change without changing the file view.

Figure 2 maps the design to the following sections. The upper region shows shared state and private changes (Section 4); the lower-left region shows native or packed metadata (Section 5). The lower-right region separates object formation from backend layout (Section 6). The mechanisms are independently optional. For example, an ML experiment can use a standalone packed dataset beside a native writable workspace.

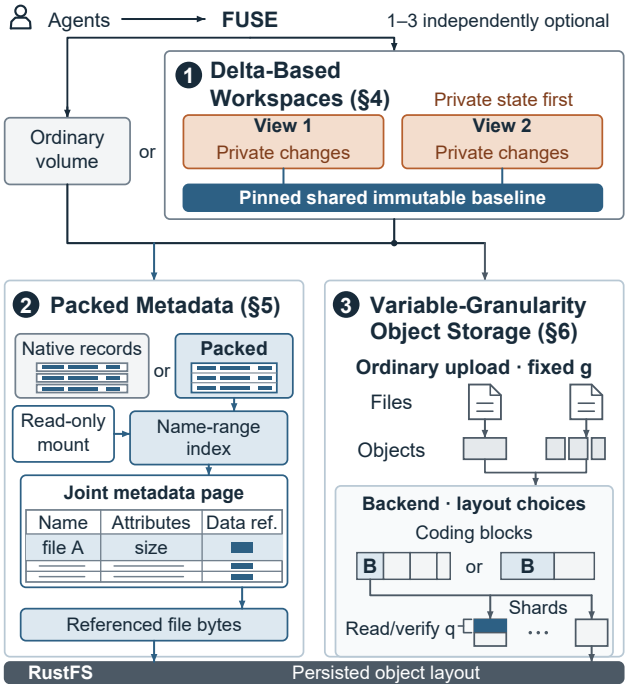


Figure 2: The three optional mechanisms and their corresponding sections. Workspaces retain private changes over shared state; Packed Metadata serves a selected read-only subtree. Object granularity separates client block size g, backend encoding block size B, and shard verification quantum q. Layouts are schematic. Implementation coverage is described in Section 7.

3.2 Pinning a baseline

A workspace binds a shared immutable lower and a private writable upper. The lower supplies a fixed version through a sealed native layer or a packed manifest. It may cover a selected subtree; the rest of the source volume need not be frozen. Resolving every lookup through a mutable latest-version pointer would violate this fixed starting view.

Private names, attributes, deletion markers, and data mappings determine which inherited records remain visible. Mutations leave the shared baseline and other workspaces' private state unchanged. Section 4 describes the rules for preserving inode identity, updating names atomically, and preventing truncated bytes from reappearing. Merging independent edits requires a separate application protocol.

3.3 Reads, writes, and publication

An inherited read obtains a locator from the lower reader and fetches its payload range. An overwrite stores new bytes and publishes a private mapping, leaving other workspaces' references intact. Buffered writes can satisfy local reads before their objects become remotely durable.

Publishing a recoverable version requires durable referenced objects and validated metadata. Ownership records retain metadata and payloads while workspaces or readers still need them. A version can contain application checkpoints, but restoring memory and open handles remains the runtime's responsibility. Section 7 describes the connected publication and recovery paths.

4 Delta-Based Workspaces

4.1 Creating a view and resolving names

Workspace creation registers an immutable base and private control state without copying the metadata tree. The catalog identifies the base by layer, sealed version, and root hash; a packed lower also binds its authenticated manifest. Creation still pays for binding validation, ownership records, and mount preparation.

Lookup first checks the private upper. An entry supplies the result; a deletion marker reports absence and stops the lookup. Otherwise, the reader consults the lower unless an opaque directory or another inheritance rule hides the name. Those rules suppress inherited entries, leaving explicit upper entries visible. Deleting an inherited name must retain its marker, since removing the upper record alone would expose the lower name again.

Directory enumeration applies the same precedence and removes duplicate names. Both the buffered and direct-I/O workspace paths preserve an inherited file's inode number when materializing private attributes or mappings. Updates through one hard-link alias therefore remain visible through the others.

The direct-I/O path keeps private namespace records in transactions. Rename locks participating inodes in number order, validates the observed records and writer authority, and commits the source deletion marker and destination binding atomically. The transaction also updates affected inode and parent records. Namespace reads validate an operation-local view and retry conflicts, so they cannot return partially applied rename records. Separate calls may observe opposite sides of the commit. The buffered path instead serializes local mutations and uses version-guarded catalog transactions. Neither provides a snapshot across multiple system calls.

4.2 Overwrites and truncation

An overwrite changes the data references for a file range. The buffered path records private extents alongside namespace and inode changes. The direct-I/O path represents each changed logical chunk with a normalized map of upper data, lower references, and zero-filled holes. An explicit map covers its applicable range completely; an uncovered interval does not implicitly refer to the lower.

A short overwrite replaces its interval and retains lower references elsewhere. Truncation must invalidate both inherited and previously written bytes. The direct-I/O path first commits the reduced size, a nonincreasing lower-visible boundary, and a cleanup intent. Cleanup replaces discarded spans with zeros; extension completes pending cleanup before increasing the size. For example, writing $[6, 10)$, shrinking to 7, and extending to 34 must leave zeros throughout $[7, 34)$. Restricting only the visible lower size would expose the old upper bytes.

The buffered client's dirty overlay serves local writes before writeback. If dirty data covers $[4, 8)$ KiB, a read of $[2, 10)$ KiB combines two committed 2 KiB ranges with the dirty 4 KiB range. Persistent private mappings and this local overlay can coexist. Their durability differs, and a short logical write need not produce a short stored object.

4.3 Publication and retention

The direct-I/O path uploads an object durably before publishing its mapping. Ordinary buffered writeback also defaults to upload-before-commit, but offers an explicit commit-before-upload mode. That mode can expose mappings before their objects are remotely available. Preparing a recoverable version must therefore drain and validate referenced objects, even when local reads already return the new data.

Worker handoff requires stopping source writes and making payloads and metadata durable before publishing the version. The destination binds that version and acquires write authority; lease and generation checks must reject stale writers. Applications checkpoint execution state and reopen files separately. Section 7 identifies which recovery steps are connected in the implementation.

Reclamation retains every object still referenced by a workspace or reader and distinguishes these objects from abandoned construction output. The metadata footprint consists of the shared baseline, each workspace's private records and mappings, and binding and ownership records. Private state grows with modifications and deletions. Physical accounting also includes payloads, encoding and checksum overhead, unreclaimed objects, and caches.

5 Packed Metadata

5.1 Serving a read-only collection

Packed Metadata targets selected read-only directories containing many small files. Even when file contents remain fixed, their names, attributes, and data locators can occupy and repeatedly query a shared metadata database. Packing moves the fixed version's read service to indexed object pages and client caches. The benefit sought is less database traffic and interference with mutable operations.

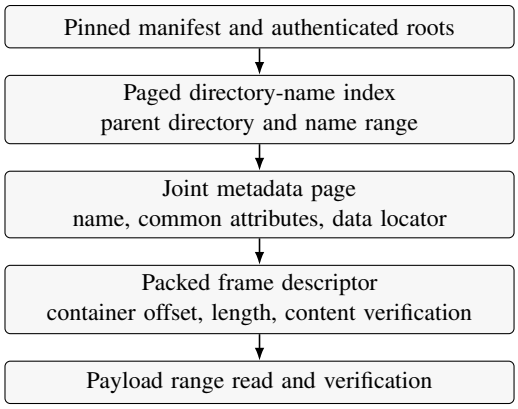


Figure 3: Packed-snapshot lookup and data access. A name lookup returns joint metadata and a data locator. A file read resolves authenticated descriptors before fetching and verifying the referenced payload range.

The packed snapshot covers only the directory version explicitly built and published. It can serve a standalone read-only mount or supply a workspace lower. Writes in the latter case go to the private upper; they never modify packed pages. Files outside the snapshot receive no benefit from this representation.

A fixed ML dataset illustrates the intended use. Several experiments can mount the same packed input while keeping code changes, checkpoints, and temporary results in writable storage. Packing is paid once per published dataset version and can be amortized across readers. Updating the source dataset requires another publication. Collections with few files, short lifetimes, or frequent changes may not repay that cost.

5.2 Grouping and indexing records

A logical read group contains entries from one parent directory over a range of raw names in byte order. Its joint metadata page co-locates each entry with common attributes and data locators. A name lookup can therefore supply information needed by subsequent attribute and data operations. Grouping neighboring names also makes the page a natural unit for reuse; Section 7 describes current caching.

Raw pages are bounded at 256 KiB. A physical container may hold several groups, so the metadata access unit need not equal the stored object. Range indexes route queries to groups; index height depends on page capacity and record count rather than source-directory depth.

5.3 Querying a pinned snapshot

A reader pins a manifest and starts from its authenticated index roots. For a name lookup, the directory-name index identifies the group covering the key. The reader resolves the

container and fetches the metadata page's byte range. Before decoding, it verifies the stored-byte digest; after decoding, it checks entry counts and name boundaries against the authenticated group reference. The matching record supplies the inode, common attributes, and data locator (Figure 3).

Other queries use indexes suited to their keys. Attribute queries addressed by inode use a separate index. Enumeration traverses ordered name ranges, with counted indexes supporting directory-ordinal navigation. A file read returns inline contents or follows a locator to an authenticated descriptor and verified payload range. Packed data frames define snapshot-level access and integrity units; the backend serves them using its own persisted object layout.

Every returned record or payload must connect through authenticated references to the pinned manifest. Missing objects and failed authentication are errors, not evidence that a name is absent. Cached records remain valid for that identity after newer versions appear. A cold lookup may still require several index requests; data reads can add descriptor and payload requests. Co-location provides opportunities for reuse without fixing the number of GETs per operation.

Supported lower queries need no per-file record in the mutable database. Private-upper checks, mutations, and ownership coordination continue through the workspace and control services.

5.4 Publication and cost

Construction starts from a consistent source view, established by a snapshot or validation protocol. Immutable output alone cannot establish source consistency. The producer groups records, uploads objects, and builds the indexed manifest. Publication then installs the binding after the referenced objects are durable and valid. Retained versions and readers keep their objects live until reclamation is safe.

A native overlay can fork after native version capture and binding. The first packed fork must additionally wait for any required construction, upload, validation, and publication. Later children can reuse the published pack. Query offload can help while the original database records remain retained; reducing their persistent footprint also requires safe retirement.

Packing trades construction and object-access work for lower database load. Nearby lookups can reuse joint pages, while isolated lookups may transfer unused records. Inlining, compression, and payload co-packing introduce additional effects that must be measured separately. The relevant cost spans construction, each use, retention, and reclamation. Long retention or rapid version churn can outweigh access savings even when queries become faster. Section 8 compares native and packed representations over this full lifecycle.

6 Variable-Granularity Object Storage

A file read names a logical byte range, but storage cost depends on the objects containing it and the units fetched and verified. Large objects can reduce request count for sequential access while increasing work for a partial read. BrewFS separates client object formation from backend encoding and verification. The finalized object connects these decisions: its length is known after assembly, compression where applicable, and framing.

6.1 From file ranges to objects

The ordinary data path maps file offsets to logical chunks and resolves the visible slices within each chunk. A chunk is an addressing range; slices identify the content supplying its visible intervals. The uploader partitions a slice using client block size g and addresses each block by slice identity and block index. A 10 MiB slice with $g = 4\,\text{MiB}$, for example, can produce raw blocks of 4, 4, and 2 MiB.

Compression and framing determine the finalized upload length L_o. Thus g partitions the slice, while L_o measures the bytes uploaded. Flush boundaries and partial writes affect object formation, and a tail need not occupy a full block. Readers use the same geometry to translate slice intervals into object references and offsets. Replacing one interval can leave untouched intervals referencing older objects.

Chunk size must be a nonzero multiple of g. Because readers reconstruct addresses from this geometry, changing g for existing data requires a compatible representation or migration. It is a configuration choice, rather than a controller that adapts to observed access.

6.2 Backend encoding and verification

Once L_o is known, the client can pass an optional encoding-block-size hint to a compatible RustFS research backend. The hint requests backend erasure-coding block size B without changing g or multipart part sizes. The backend checks eligibility and compatibility, then persists the effective B. Omitting a hint retains default selection. Multipart initiation uses the complete object's length, and retries retain that object's decision.

The interface accepts fixed hints of 64 KiB, 256 KiB, 1 MiB, and 4 MiB, or a length-based policy. These choices apply to the formed object. Section 7 describes the current policy and its relationship to the backend default.

The physical verification/read quantum q determines how much shard data is verified together. A separate backend policy can select smaller units for eligible new framing. Full stripes must contain a whole number of verification units per shard, with coding blocks aligned across all data shards. Hint admission and new verification framing have separate

g	Object count	Whole-object bytes fetched
1 MiB	64	2 MiB
4 MiB	16	4 MiB
8 MiB	8	8 MiB
16 MiB	4	16 MiB

Table 1: Arithmetic example of object formation and a 2 MiB partial read. Complete intersecting objects are fetched, with no caching or compression and ignoring framing overhead. Range reads can transfer less.

compatibility gates. Reads and repair use persisted geometry even after write settings change.

6.3 Physical work for a partial read

Range GET can retrieve part of an object when the format permits independent access. The ordinary store supports page-aligned range reads for eligible small requests; compressed blocks or whole-object fetching can require larger transfers. Packed frames provide a further decoding and integrity boundary inside snapshot objects. Their size is distinct from both g and q. Caching and concurrent-read coalescing determine which requests reach the backend.

The backend maps an object range to shard intervals and rounds each interval outward to verification-unit boundaries. A 2 KiB shard read at offset 3 KiB, for example, touches two 4 KiB units: 8 KiB plus checksums, before tails or batching. Client fetching, compression, and coding determine the shard interval. Smaller q can reduce over-read but adds checksum overhead. With unchanged data and parity shard counts, nominal redundancy remains constant; a large B does not imply a fully padded tail stripe.

6.4 Granularity tradeoffs

Table 1 illustrates the tradeoff in g for an uncompressed 64 MiB slice starting at offset zero. Increasing g reduces object count but increases transfer for a read of $[10, 12)$ MiB when complete objects must be fetched.

These layers require distinct measurements. Network read amplification compares object-response bytes with application bytes served; physical read amplification uses storage-device bytes as the numerator. Upload amplification compares uploaded bytes with application bytes written. Request count, compression work, and cache occupancy capture additional costs. Compaction, retries, and short slices can increase upload work, so smaller objects need not reduce write amplification.

A smaller backend quantum helps only if client fetching exposes sufficiently small ranges. Snapshot producers and workspace upper writers also have their own upload formats. The implementation differences below determine which combinations can be compared without changing the client path.

7 Implementation

BrewFS connects the mechanisms through buffered, direct-I/O, and standalone read-only paths. Their cache, publication, and recovery behavior differs. The following describes the inspected implementation; runtime and failure validation is included in the evaluation plan.

7.1 Access paths

An ordinary volume resolves mutable metadata and uses buffered caching and writeback. A workspace can reuse that path with private-state resolution over a sealed catalog or packed lower. A standalone packed reader serves read-only access without a private upper. Its metadata cache is separate from payload caching; configurable payload-cache and prefetch budgets are not currently admitted on this path.

The packed reader caches index pages, resolved names (including absence), and attributes. An uncached name still fetches and decodes its joint metadata page. Reusing decoded pages across different names remains an optimization opportunity; the current result caches do not establish that benefit.

The direct-I/O workspace stores private namespace and mapping records transactionally. It implements the rename and truncation protocols in Section 4, bypassing buffered writeback and metadata time-to-live caching. Admission bounds limit operation sizes, complete-directory enumeration, and the live ownership set. This path currently supports newly created workspaces without process-crash takeover.

Direct reads authenticate an entire upper object before returning a subrange. A smaller backend verification unit may therefore leave short-read work unchanged. Comparisons with buffered volumes must account for this fetch policy as well as the different caches.

7.2 Publication and recovery

Snapshot publication validates uploaded objects before installing the lower binding and acquiring reader ownership. Recovery also checks generations and leases; a manifest identity alone conveys no write authority. The connected recovery path reuses the same persistent storage allocation and checks worker identity and scoped metadata authority. Arbitrary-worker migration still requires orchestration.

Catalog lifecycle operations support quiescence, object verification, retention, and atomic version transitions. A runtime must connect these operations to handoff, checkpoint execution state, and reopen tools. The recoverable storage state is a published file version.

7.3 Layout support

The ordinary uploader defaults to $g = 4\,\text{MiB}$ within 64 MiB chunks, and readers use the same geometry. Incompatible configurations are rejected. The proposed 1/4/8/16 MiB settings are fixed configurations.

Backend hints require compatible client and server support. Hint admission and new verification framing have separate enable and fleet-compatibility gates, both disabled by default. Effective layouts are persisted for later reads and repair; changing settings does not re-encode existing objects.

The ordinary uploader propagates hints. Direct workspace writes and the current snapshot producer use defaults; extending them must preserve conditional object creation and conflict rejection. Direct workspace objects contain write fragments, capped at the automatic policy's no-hint threshold, rather than ordinary buffered blocks.

The current policy omits hints for $L_o \leq 1\,\text{MiB}$ and requests $B = 1\,\text{MiB}$ otherwise. The inspected backend already defaults to $B = 1\,\text{MiB}$, so enabling the policy can leave stored geometry unchanged. It can nevertheless change client fetch and cache behavior. Experiments must inspect actual (g, L_o, B, q) and separate these client effects from changes in physical layout.

8 Evaluation Plan

The planned evaluation isolates each mechanism before testing fork, migration, and concurrent execution. It measures storage service to agents inside running containers, including access to shared read-only inputs. System-performance results are pending. The eight-run characterization in Section 2.3 and the seven later replay archives are separate datasets.

8.1 Systems, inputs, and accounting

JuiceFS is the external distributed-file-system baseline [8], using native cloning and shared read-only directories where appropriate. Both systems use the same four-server object backend at separate times, with isolated namespaces and matched client, metadata, cache, and persistence budgets. Execution freezes software revisions and qualifies supported operations and restart semantics. Research-backend extensions are reported separately.

Inputs include a 1 GiB tree with 32,896 files, a 1 GiB indexed record collection, and trees containing 32,768 or 1,048,576 one-KiB files. A larger small-file case is bounded by available resources. Seven complete traces cover code tasks, build/test, and archive/indexing work. Two capture the same Pytest task; replay copies add load but no task diversity. The companion protocol fixes generators, queries, branch trees, arrivals, deadlines, and repetitions.

Accounting spans preparation, execution, validation, retention, and reclamation, including failed and timed-out attempts. Shared work and object residence are counted once across uses and versions. CPU time, transferred bytes, and monetary cost retain separate units; storage charges include residence after the last use. Batch duration is measured directly rather than summed from concurrent tasks. File replay and actual agent sessions are evaluated separately.

8.2 Overlay fork and retained branches

RQ1. How much state and work does overlay fork save without packing metadata? A common operation suite compares an ordinary volume, native-metadata overlay, and packed-metadata overlay at fixed object layout and payload caching. Lookup, read, write, and sync tests expose steady-state overhead. A volume lacking fork uses materialization to produce the same isolated child state, including its preparation cost.

A parent changes data and names, publishes a version, and forks 1, 16, or 64 children. Each child modifies and saves private state. Readback checks inheritance and parent/sibling isolation; clean-baseline creation is a separate case. A 32-fork budget compares a star, a four-way tree, and four chains of depth eight, retaining 33 states in each case.

An internal control materializes metadata under the same API, payload-sharing, cache, and durability contract. JuiceFS can retain native block sharing. Measurements cover parent preparation, first correct child operation, first durable edit, save latency, private metadata, and uploaded bytes. Retention tests create 1,000 and 10,000 recoverable states, with 100,000 attempted under fixed time and memory limits. Reopening states and deleting half tests retention and reclamation. Admission failures and outstanding cleanup are reported.

8.3 Read-only metadata service

RQ2. When does packing a fixed small-file collection reduce database load enough to repay construction and publication? The primary comparison holds an overlay and its content fixed while changing the lower from native to packed metadata. Both start from a consistent version. Standalone read-only mounts isolate representation without requiring a workspace.

We time native version capture and binding, the additional work needed for the first packed fork, and later forks of each published version. Packed preparation includes construction, upload, validation, and publication. Time to the first useful operation and full-batch duration are measured directly, allowing phases to overlap.

Each small-file query session contains 32,768 operations. Workloads cover random positive lookups, mixed positive/negative lookups, inode attributes, neighboring names, enumeration followed by attributes, and lookup followed by

a read. One, 16, or 64 consumers access overlapping or disjoint paths. Cold, index-warm, and metadata-warm cases declare equal supported cache budgets. Representation controls disable inlining and compression; practical configurations measure their additional effects.

Counters distinguish lower-database, private-upper, and control requests from index, joint-page, descriptor, and payload requests. Measurements include query latency, database CPU and memory, persistent bytes, transfers, decoding, verification, and client and object-service resources. A fixed-rate mutable-workspace probe shares the metadata service with identical background schedules against native or packed inputs. Its queue, deadline completion, and tail latency test whether offloading read-only queries protects active updates.

An ML-oriented case separates dataset discovery from sample access. Workers enumerate a fixed directory collection, then open samples under controlled sequential and shuffled orders. Fresh sessions and persistent sample lists distinguish repeated discovery from steady loading. Writable outputs remain outside the packed dataset. The primary metrics are metadata service work and interference with concurrent writers; loader waiting and training throughput require separate end-to-end measurements. Larger-file collections test how the opportunity changes as per-file metadata becomes less prominent.

Read-only layout comparisons use a compatible lazy reader, such as Nydus or SOCI, at matched content, integrity, cache, and payload settings. They measure construction, version updates, memory, transfers, and sustained query service. A pre-extracted local image is a placement comparison, with local storage charged. An equivalent immutable sorted-KV control would isolate joint-page organization, without reproducing DeltaFS.

Each version serves 1, 4, 16, 64, or 256 query sessions. Eight version cycles change 0.1%, 1%, or 10% of files, without assuming an incremental packer. Post-use retention is 0 or 600 seconds. At equal reuse and retention, resource-specific costs identify observed break-even regions. Source records retained in the database are distinguished from safely retired records. Rapid updates, weak locality, or warm native caches may prevent break-even within the measured range.

8.4 Object formation and backend work

RQ3. How do client and backend granularity affect physical work, write completion, and repair? Identical writes first compare supported $g = 1, 4, 8, 16\,\text{MiB}$ configurations at fixed backend geometry. Fixed objects and S3 requests then compare admitted B hints and q policies. Range and whole-object fetching are compared separately, controlling caches, coalescing, and prefetch. Persisted layouts determine whether a policy actually changed geometry.

Indexed 4 KiB reads, shared and disjoint keys, boundary-crossing reads, scans, overwrites, and supported degraded reads exercise different tradeoffs. Object counts, finalized lengths, HTTP and device-read bytes, checksum and encoding CPU, remote-write completion, and repair traffic explain the resulting latency. Missing physical counters are reported; HTTP bytes cannot substitute for device reads.

8.5 Migration and recovery

RQ4. How much work does migration require before and after publication? Sixteen workspaces run on four source workers, with four target workers already available. We migrate 0, 1, 4, or 8 workspaces while the others continue; a larger configuration migrates subsets of 256 workspaces. Native and packed paths have paired no-migration controls.

Handoff occurs at a completed-tool boundary, with either a published version or pending modifications. Pending work includes draining, capture, and required packing and publication after the request. Queueing counts toward interruption. JuiceFS reopens its remote directory without forced cloning. Cold and warm targets measure authority transfer, first correct operation, full continuation cost, transfers, and neighboring tasks' deadline completion and tail latency. Recorded node identities distinguish cross-node migration from container handoff.

Failure tests interrupt before and after publication, disconnect source coordination while retaining storage access, and fail the recovering target. Independent readback checks committed data and attempted stale-writer changes. Process termination, application checks, and storage-enforced rejection are recorded separately. A synthetic state file tests file-state handoff. Actual agent tests preserve history and remaining budget across fork or migration, and require a subsequent valid tool result and final task grading.

8.6 Concurrent tasks and analysis

RQ5. Do component savings increase useful capacity at fixed resources? The fixed-work panel assigns eight copies of each of seven traces: 56 assignments at concurrency 1, 4, or 16 on four workers. A common deadline includes queueing. A separate scale study uses 4, 16, 64, or 128 workers, four slots and seven assigned traces per worker, against fixed backend capacity. Its growing workload measures throughput and per-case latency. Node placement and replay-engine CPU limits are recorded.

Internal comparisons change one supported mechanism within a matched path. Composition is compared with its best supported proper subset; component gains need not imply a positive interaction. External system comparisons are reported separately. Three initial independent paired batches per cell provide raw outcomes and ranges. Stronger inference requires a sample size fixed from a separate variance pilot; individual requests and sibling tasks are not independent replicates. Failures remain in assigned-work denomina-

tors. Complete-success makespan is reported only when all assignments succeed, alongside deadline completion, failed outcomes, and measurement gaps.

9 Related Work

Private workspaces. OverlayFS combines upper and lower trees using copy-up and deletion markers [13]. BranchFS adds a fork/explore/commit lifecycle with first-commit-wins resolution; its evaluated prototype copies an inherited file when it is first modified [22]. BrewFS retains private namespace changes and data-range mappings over a pinned baseline, preserving references to unchanged ranges. Its lower-query interface accepts native or packed metadata. YoloFS emphasizes staging, reversible effects, and permissions [29]; AgentFS combines files, key-value state, and tool-call records in SQLite [4]. These execution-control and observability functions complement the storage mechanisms studied here.

Mutable metadata services. TableFS and IndexFS use table-based layouts for metadata and small files; IndexFS also partitions directories and supports caching and bulk insertion [18, 19]. LocoFS and InfiniFS separate metadata responsibilities to improve distribution and access [12, 14]. Mantle targets hierarchical object metadata, while SwitchFS coordinates asynchronous directory updates [11,26]. Packed Metadata changes the service path for a selected fixed subtree. Supported queries use object pages and client caches, while mutable work and lifetime control retain their existing services. The question is whether this separation reduces interference enough to repay publication and access costs.

Private namespaces and immutable logs. DeltaFS combines private namespaces with object-backed metadata. Jobs publish mutation logs as SSTables; subsequent jobs select and, when needed, merge snapshots. Job-local servers and compaction reduce dependence on a globally synchronized metadata service [28]. BrewFS instead maintains a current-value private upper over a pinned lower. Packing separately organizes fixed records by directory and name range, co-locating attributes and data locators. A native-metadata workspace can therefore fork without constructing that representation. The distinction concerns update representation, query organization, and when construction cost is paid.

Read-only distribution and indexing. CernVM-FS publishes content-addressed trees through nested SQLite catalogs and downloads metadata and data on demand [2]. It is a close precedent for serving large immutable collections outside a mutable metadata service. OCI defines layered filesystem archives, including compressed tar representations [16]. Lazy loading is supported by systems such as SOCI [20]. Nydus separates filesystem metadata from chunked data and provides caching and integrity verification [15]. Slacker and DADI also reduce image distribution and startup work through demand-driven access [5, 10].

CoFS exploits a fixed image namespace by building minimal perfect hashes and storing metadata in a dense array. Kernel lookup and cached-data paths reduce userspace interposition [23]. BrewFS addresses the metadata service used by agents throughout execution. It offloads queries for selected fixed subtrees while retaining mutable workspace operations in their existing services. Its packed representation groups directory entries, attributes, and data locators into bounded pages reached through authenticated range indexes. These groups define remote metadata access units independently of their physical containers. The relevant comparison with immutable distributions is the work required to locate, fetch, retain, and reuse these records.

Object granularity. Ceph and Tectonic separate metadata and storage services [17,25]. Azure's erasure coding reduces repair traffic, Morph studies redundancy transitions over a file's lifetime, and Okapi separates striping from redundancy grouping [1, 6, 9]. BrewFS connects object formation to optional backend layout choices. Client block size g controls formation, backend block size B controls encoding, and q controls shard verification and read coverage. Finalized object length informs a hint; persisted geometry governs reads and repair. The design retains the backend's erasure code and exposes these distinct units to the filesystem. Its benefit depends on the ranges actually requested by the client.

10 Conclusion

BrewFS provides independently optional mechanisms for private workspace state, read-only metadata, and object granularity. The coding-agent traces motivate evaluating access costs alongside fork latency. The remaining evaluation must determine when metadata offload and selective reads repay construction and retention across concurrent agent tasks.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC application software and conditions databases using the

CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[4] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[5] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[6] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[8] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[9] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[10] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[11] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[12] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[13] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[14] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[15] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[16] Open Container Initiative. Image layer filesystem changeset. OCI Image Format Specification. Accessed 2026-10-26.

[17] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[18] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[19] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[20] SOCI Snapshotter Contributors. SOCI snapshotter. Official project documentation. Accessed 2026-10-26.

[21] Torchvision Contributors. DatasetFolder and ImageFolder. Torchvision documentation and implementation. Accessed 2026-10-26.

[22] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[23] Li Wang, Jinxu Du, Yang Yang, Qingbo Wu, Tao Liu, and Haoze Wu. CoFS: A filesystem for fast container startup. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 415–423. USENIX Association, 2026.

[24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[25] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[26] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[28] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[29] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.